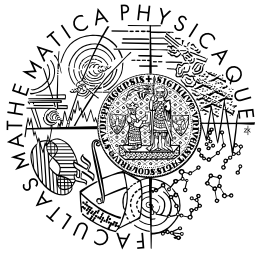

FACULTY
OF MATHEMATICS
AND PHYSICS
Charles University

BACHELOR THESIS

Adam Balko

Arduino platform simulator

Department of Software and Computer Science Education

Supervisor of the bachelor thesis: RNDr. David Bednárek, Ph.D.

Study programme: Computer Graphics, Vision and Computer Games Development

Prague 2026

I declare that I carried out this bachelor thesis on my own, and only with the cited sources, literature and other professional sources. I understand that my work relates to the rights and obligations under the Act No. 121/2000 Sb., the Copyright Act, as amended, in particular the fact that the Charles University has the right to conclude a license agreement on the use of this work as a school work pursuant to Section 60 subsection 1 of the Copyright Act.

In ..... date ....

Author's signature

Dedication.

Title: Arduino platform simulator

Author: Adam Balko

Department: Department of Software and Computer Science Education

Supervisor: RNDr. David Bednárek, Ph.D., Department of Software Engineering

Abstract: This thesis focuses simulating code of Arduino microcomputers in a virtual environment with a graphical user interface without the need of emulation. An important part of the thesis shows how and why should be the simulator and the GUI split into two processes, and how to keep them synchronised. This introduces the reader into the field of interprocess communication and its implementations using TCP connection, shared memory and events. Another important part focuses on simulation of digital circuits with extensible component library. It describes how new components and circuits can be created from YAML, SVG and C# files by the user.

Keywords: digital circuit simulator, Arduino

Název práce: Simulátor prostředí Arduino

Autor: Adam Balko

Katedra: Katedra softwaru a výuky informatiky

Vedoucí bakalářské práce: RNDr. David Bednárek, Ph.D., Katedra softwarového inženýrství

Abstrakt: Tato práce se zaměřuje na simulaci kódu mikropočítačů Arduino ve virtuálním prostředí s grafickým uživatelským rozhraním bez nutnosti emulace. Důležitá část práce ukazuje, jak a proč by měly být simulátor a GUI rozděleny do dvou procesů a jak je udržovat synchronizované. To čtenáře uvádí do oblasti meziprocesové komunikace a její implementace pomocí TCP spojení, sdílené paměti a událostí. Další významná část se zaměřuje na simulaci digitálních obvodů s rozšiřitelnou knihovnou komponent. Popisuje, jak může uživatel vytvářet nové komponenty a obvody ze souborů YAML, SVG a C#.

Klíčová slova: simulátor číslicových obvodů, Arduino

Contents

Introduction

Arduino is a very popular microcomputer among amateurs and professionals alike. Despite this, testing the code for an Arduino machine can be fairly difficult. Debugging options are limited and the user needs to rely on additional measuring equipment. The usual choice is to test the code with an AVR emulator. Emulation is the process of running machine code on a hardware that doesn't match its instruction set. AVR is the processor that controls the Arduino board. These emulators are sometimes part of a bigger logical circuit simulation system, other times they are just a single piece of software.

Goal of this work is to skip the emulation process and create a working Arduino simulator by mimicking the changes of digital values on the board's pinout. The user code will be compiled directly to the x86-64 architecture instruction set, which is our target platform, and thus able to run at the native speed of the host machine.

Bundled with this simulator is a graphic application for creating and testing logical circuits, with Arduino as it's main component. The point of this application is to attach to the running Arduino simulator and visualize the expected Arduino behavior with user's code on a given circuit. We want to give the user the power to compose their own circuits, and define the behavior and looks of their components. Additionally, the user will be able to debug their program, while the GUI is running, using their preferred IDE and C++ debugger, providing real-time feedback and inspection when stepping through the code.

1 Introduction to Arduino programming

1.1 Sketches

The Arduino programs are commonly referred to as *sketches*. A sketch is not a single file, but an entire directory, usually populated by files with the `.ino` extension, contaning the source code of the program. `.cpp`, `.c`, `.S` (assembly files), and header files like `.h` are also accepted as files contaning source code. Every sketch must contain a `.ino` file with a file name matching the sketch root directory name [1].

The Arduino programming language, in which `.ino` files are written, is a C++ dialect with very subtle differences, mostly concerning linking of files and ommitting function declarations. Bodies of function definitions and variable declarations can be considered as a perfectly valid C/C++ code.

The essential functions that an Arduino programmer has to define in their sketch are `void setup()` and `void loop()`. The `setup()` function is called once, when the program is booted. That is when the Arduino board is powered up, the program is uploaded to it, or when the reset button is pressed. The `loop()` function is called in a loop indefinitely. This is the proper place to define any interations with the board, detecting inputs, updating timers, etc.

1.2 Arduino board interface

The target board `Arduino Uno R3 SMD` provides 6 analog and 14 digital general purpose pins, some of which can be used for I2C and SPI protocol communication, or PWM modulation. The most simple to use and simulate are the digital pins for general purpose I/O, so this thesis focuses exlucively on using these pins as such [2].

The digital pins can be manipulated using only these 3 functions:

- `void pinMode(pin, mode)` - configures specified pin either as an input or an output
- `value digitalRead(pin)` - reads the voltage on the pin, stores it as a digitla value, either 1 or 0
- `void digitalWrite(pin, value)` - when set to output, sets the voltage of the pin to corresponding value. If set to input, current driven components may not work correctly, since the current is suppressed by an internal pull-up resistor.

Pins are identified by their index. `mode` and `value`'s underlying type is also an integer, but it is preferred to use predefined constants, such as `INPUT/OUTPUT`, `HIGH/LOW` and such. Below is an example of a sketch from the Arduino's documentation, using only the described functions:

Listing 1.1 Sketch example [3]

```
int ledPin = 13;  // LED connected to digital pin 13
int inPin = 7;    // pushbutton connected to digital pin 7
int val = LOW;    // variable to store the read value

void setup() {
  pinMode(ledPin, OUTPUT);  // sets the digital pin 13 as output
  pinMode(inPin, INPUT);    // sets the digital pin 7 as input
}

void loop() {
  val = digitalRead(inPin);   // read the input pin
  digitalWrite(ledPin, val);  // sets LED to the button's value
}
```

1.3 Build process

Before the code is compiled, all `.ino` go through a simple preprocessing stage. `#line` directives are added. These link to the original line and file, when the code is debugged. All files are concatenated together, function declarations are generated at the top, and the `Arduino.h` is included. This header file is crucial as it declares the `void` and `loop` functions, functions for manipulating pins, common constants and mathematical functions, etc. At this point the sketch becomes a complete C++ source code.

It is also at this where we steer away from the standard compilation process, and apply our own `Arduino.h` implementation in our simulator. Usually, the C++ code would be linked to an implementation that contains binaries and constants for the given microchip installed on the target board. Then it would be compiled with a compiler for the given instruction set (e.g. `avr-gcc`). The code would be then transmitted over USB to the user's Arduino.

In our case, we include the entire preprocessed sketch in our simulator project written in C++, and compile it as whole for the instruction set of the host machine. This executable will implement mock functions declared in the `Arduino.h`, arduino's internal state, and a way to communicate the change of state to an external GUI application.

The entire process from preprocessing to compilation, can be orchestrated with the use of `arduino-cli` command line application. We will use it to preprocess the `.ino` for us, so that we end up with a clean C++ file we can include in our project.

1.4 Specific goals of the thesis

Target platforms:

- Linux (x86-64)
- Windows 10 or higher (64-bit)

Software prerequisits:

- Minimal system requirements of Avalonia UI
- arduino-cli
- .NET 9 SDK or higher
- C++ compiler
- CMake
- C++ Boost libraries:
  - asio
  - interprocess
  - program_options

Simulator functional requirements

- Defines platform independent constants and macros from `Arduino.h`.
- Implements `digitalWrite`, `digitalRead`, `pinMode` and `millis` functions from `Arduino.h`.
- Declares and calls `setup` and `loop` functions with the behavior expected on an Arduino machine.
- Compiles and runs any valid `Arduino UNO R3 SMD` sketch, preprocessed with the `arduino-cli` tool, containing only symbols mentioned above.
- Reflects behavior of the above functions in an internal Arduino state. Logs changes of this state in a text file.
- Integrates an interprocess communication mechanism for exchanging data with an external GUI application.
- The events incoming from a GUI application must be resolved in a manner, that preserves the continuity of events.

GUI functional requirements

- The application must stay responsive during debugging of the simulator process.
- The application must be able to connect to the interprocess mechanism of the simulator.
- Simulates simplified digital circuits. Exact physical properties, like voltages and currents might not be implemented. Built-in components may be implicitly connected to ground.

- Implements mechanism for defining custom components and visuals with `.yaml`, `.cs` and `.svg` files. These components include interactive controls, such as buttons.
- Implements mechanism for defining a circuit (scene) using the custom components.
- Predefines LED, button, voltage source and Arduino components, and a scene which utilizes these components.
- The changes to the simulator's internal state must be immediately reflected on visual components connected to the Arduino in the scene.
- The scene is able to be zoomed and panned. The visuals of the components must be rendered using vector graphics.
- Multiple interactable components may be pressed at once, with behavior expected from the source code of the sketch.

2 Architecture design

2.1 Architecture overview

The whole project is divided into two main parts. The purpose of the backend is to execute the Arduino code and simulate the change on the board's pinout. The purpose of the frontend is to reflect these changes on a user defined logical circuit, while offering the user control over the simulation. We chose to implement the backend in C++ and the frontend in C#. Both backend and frontend are running in its own separate process.

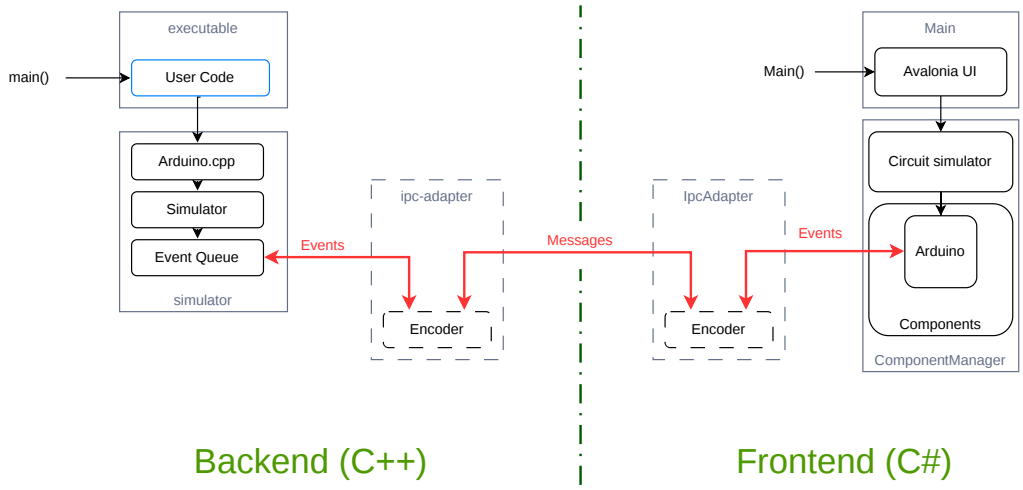

Figure 2.1 Top-level architecture overview

The Figure 2.1 shows the most important systems in each process. The user code is compiled together with the `executable` module of the backend. The declarations in the header `Arduino.h` prepended to the top of the `.ino` is implemented by a mock library `Arduino.cpp`. For each observable action an event is created and passed to the singleton `Simulator` class to compute and put into an event queue. The event is also communicated to the frontend process.

In the frontend the events are consumed by the `Arduino` component, one of several components predefined as part of the `ComponentManager` project. This project is responsible for loading user-defined components and scenes, which connect the components together in a circuit. The circuit is simulated in the same project. The user interface is implemented in the `Main` project using Avalonia UI as a framework. Interacting with the UI may invoke new events in the `Arduino` component. Those are sent back to the backend's event queue in the same manner as they were received.

The IPC Adapter is an interface implemented in each process responsible for encoding and decoding the events into a standardised format and relaying to the other process.

2.2 Inter-process communication

Implementing processes that depend on each other comes with certain challenges. Processes always run concurrently, so a variety of synchronization techniques must be applied to avoid race conditions. Before that we need to determine how will the processes exchange data. Process is a kernel level mechanism, thus approaches to handle the exchange can vary between operating systems. These mechanisms are what is called *Inter-process communication*, or IPC for short.

2.2.1 Considerations

There is a technical and an engineering reason for splitting the work into more than one process.

The user must be able to have full control over the frontend user interface at all times, even while the Arduino code is being debugged. Every common debugger blocks all running threads of a process, when a breakpoint is hit. elaborate If the GUI has been part of the same process as the systems running the simulation, it would freeze and become unresponsive every time a breakpoint is hit. The only way to avoid this behavior is to separate the GUI to a different process. We want to support breakpoints in our project, as they are the most useful tool in a debugger's toolkit.

Splitting the application into multiple processes is actually a common practice among larger pieces of software. The program of each process can be written in a different programming language and thus delegated into teams with different competencies. They mark clear separation of concerns and force a solid interface design. Each process is easily swappable for another implementation with the same interface while the other processes are still running. In our case, the GUI process could be replaced by a web client, a CLI client, or an IDE add-on.

There are more than a few ways to handle IPC on every OS. The ones that we implemented are *TCP stream* and *shared memory*. These are among the most common approaches available both on Windows and Linux. In either case, we must clearly define what the shared data represent. Although shared memory has more flexibility than TCP, we stuck to sending discrete text or binary messages of predetermined format over time, to keep the implementation simpler and the two approaches interchangable.

Another question is what will we encode in our message. At first we encoded the full state of the simulated Arduino in *snapshots* but due to its various shortcomings we later switched to sending discrete *events*. So the simulator in the backend and its interpratation in the frontend both operate in terms of events, but TCP and shared memory modules only understand the notion of sending and receiving messages with very distinct interfaces. That's why we need to design an adapter in both processes and for both IPC methods that sets up the mechanism and encodes or decodes the passing messages into structured event data the simulation logic understands. The IPC adapter then informs the other systems in it's process any time a new event is received.

2.2.2 Snapshotting

In the early iterations, a snapshot of the Arduino state was sent over one message. That included the pinout state, pin modes, timer, etc. The state was periodically requested by the client with a *Read* message without payload. When a new value is read on an input pin in the frontend, a *Write* message with the state as payload would be sent to the server and the values of input pins would be overwritten atomically to the backend's internal state.

This approach was easy to implement but flawed. The main issue was the frequency of reads. At lower frequencies, the system is prone to temporal aliasing. Find temporal aliasing definition. Some events, like very short impulses, can be missed completely. This is not much of an issue in purely combinational circuits. But we want our frontend to support stateful components, e.g. shift registers present on the Funshield, which are controlled by digital writes less than a microsecond a part.

Highering the frequency to such an extent is not viable. elaborate

2.2.3 Events

Right now synchronization between processes is maintained by recording every significant event produced by either the backend or the frontend, and sending it to the other process. Each process is equipped with an event queue, that consumes the events in order of the least recent to the most recent. Consuming an event means invoking the appropriate function based on the type of the event with arguments stored in the event's signature.

The Arduino object in the frontend has no logic on its own, so the only consumed events are those produced by the backend. For that reason a simple queue is sufficient, and no further synchronization is necessary. In the backend however, both simulator's and frontend's events are consumed. Sending an event to the frontend and receiving direct consequeunces of the event back can involve a significant time overhead. In the meantime the simulator produces more events that are consumed almost instantaneously. We need to ensure that the events are consumed in a valid order. We also need a robust queue implementation that isn't prone to race conditions.

An event is labeled with two timestamps. Timestamps don't have to be based on a real clock, the only requirement is that it is a non-decreasing sequence of numbers. The first timestamp sent is the simulation time, the time as perceived by the Arduino being simulated. That is time given by a clock that is started right before the `setup()` is invoked. It can be manipulated by the user by pausing or slowing down. This is to aid the user while analyzing the logs of the simulator.

The other timestamp included is *Local virtual time* (LVT)[1]. Each process holds its own integer value of LVT. This number is increased with newly produced events and events received from the other process. When a decision has to be made on which event to consume next (like in the case of the backend's queue), the event with lower LVT is chosen. The relation $LVT_1 > LVT_2$ means that the event with LVT_1 happens after the event with LVT_2. In other words, event 1 is the consequence of the event 2 regardless of the real time passed between the events.

The rest of the information stored in an event is its type and arguments. The type corresponds to the action taken after the event is consumed, like writing to a pin or changing pin mode. Each type can define up to 3 integer parameters, e.g for a *write* event that is id of the pin, and a digital value we want to write to it. By limiting the number of arguments, we intend to avoid heap allocation of these events, which could considerably slow down the simulation with programs that produce a lot of events in a short time.

There are many ways to encode such an event into a message. Currently the only encoder implemented in this work is the text encoder, which parses values of events into human readable strings separated by a colon. The messages themselves are separated by a semicolon. This particular encoding is quite space ineffective, but it helps a lot during debugging of the IPC systems.

2.2.4 TCP stream

Messages can be sent and received through a stream of data on a TCP connection. TCP is commonly used to send data over a network, meaning backend and frontend processes can run on different machines. But listening to a connection on the same machine on the same port works just as well. TCP messages are guaranteed to arrive in the same order as they were sent.

The backend implements the TCP server, while the frontend implements the client. The implementations and design considerations behind these are very different based on the language they were written. But in both cases TCP handlers need to run on their own threads, due to how heavyweight elaborate they are. There is a buffer to which read bytes are stored from an OS network stream. These are passed as a chunk to a higher level message handler that splits the chunk into individual messages according to predetermined delimeter character. When a message needs to be sent from another thread, it is passed to the message handler where it is posted to the end of a queue and is scheduled to be processed by the OS.

There is a minor limitation of IPC over TCP that makes the debugging experience of the user's code uncomfortable. As we established, hitting a breakpoint will block all running threads of the process, including the thread with the TCP handler. When stepping through the lines of the code where events are produced, the TCP thread has to schedule posting of the message with the event, and it takes time for it to be eventually sent. In many cases the TCP thread won't be fast enough to send the message in the time the main thread progresses to the next line where a breakpoint will be hit again. As a result, consequences of the event won't be reflected in the frontend process until another one or more lines have been stepped through. This is the main reason we decided to also implement IPC with memory mapping, even though TCP is fully functional.

[1]The terminology and principles are inspired by the *Optimistic synchronization* paradigm in distributed systems. In this paradigm, the events are consumed as fast as possible, and then rolled back once an event with a lower LVT than the ones consumed appears (so called *Time Warp* mechanism). But Time Warp is not viable for simulating code of a high level language like C++ without custom memory management, and in our implementation it isn't even necessary. [4]

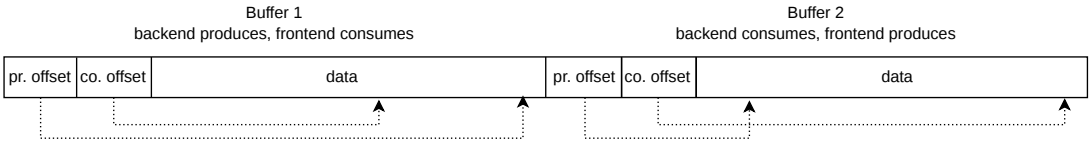

Figure 2.2 Memory layout of the shared memory region

2.2.5 Shared memory

Memory mapping is the functionality of an operating system to relate a region of main memory 1:1 to a region in the address space of a process. This is an important idea in establishing a virtual memory of a process, but OS objects can be mapped as well. It could be either a *memory-mapped file*, contents of a disk file moved to the RAM. Or it could be a special OS object called *shared memory*, living in the kernel heap, specifically designed for IPC. In both cases, multiple processes can map the same memory region to their own address space. Thus they have access to the same data.

The terminology and API differ significantly between Linux and Windows, but with the use of the right libraries, the differences can be abstracted away and used as a single concept. So from now on we refer to the idea of mapping the same memory region to multiple address spaces as *shared memory*.

This tool is very powerful, we can address the shared data by pointers as we would with any other data of the process. There is also no need for system calls once the the memory has been mapped. No special instructions need to be used for reading or writing and all changes are immediately visible on all of the processes that map the same region. As long as we do the writes in the simulating thread, we avoid the problem with delayed observations during debugging we encountered with TCP.

In the shared memory region, we establish a data structure that stores messages in a sequential order. We opted for a circular buffer to fully utilize the limited space of the region. There are actually two non-overlapping buffers in the same region together with a few bytes of metadata about their current state. A process consumes messages from one buffer while producing new messages to another. This is to avoid numerous race conditions that would occur while writing and reading from the same buffer while keep the synchronization mechanism lock-free.

As for the metadata, for each buffer a position after the last consumed and produced message is stored. It is important to never store references as pointers in the shared memory itself. The memory region will be most likely mapped to different addresses in each process. Because of that the *producer offset* and *consumer offset* are regular integers denoting the distance from the first adress of the buffer in bytes. When the offsets are equal, it means the buffer is empty. We don't care about concurrent updates of these offsets, one process changes only one of the offsets for the duration of the simulation.

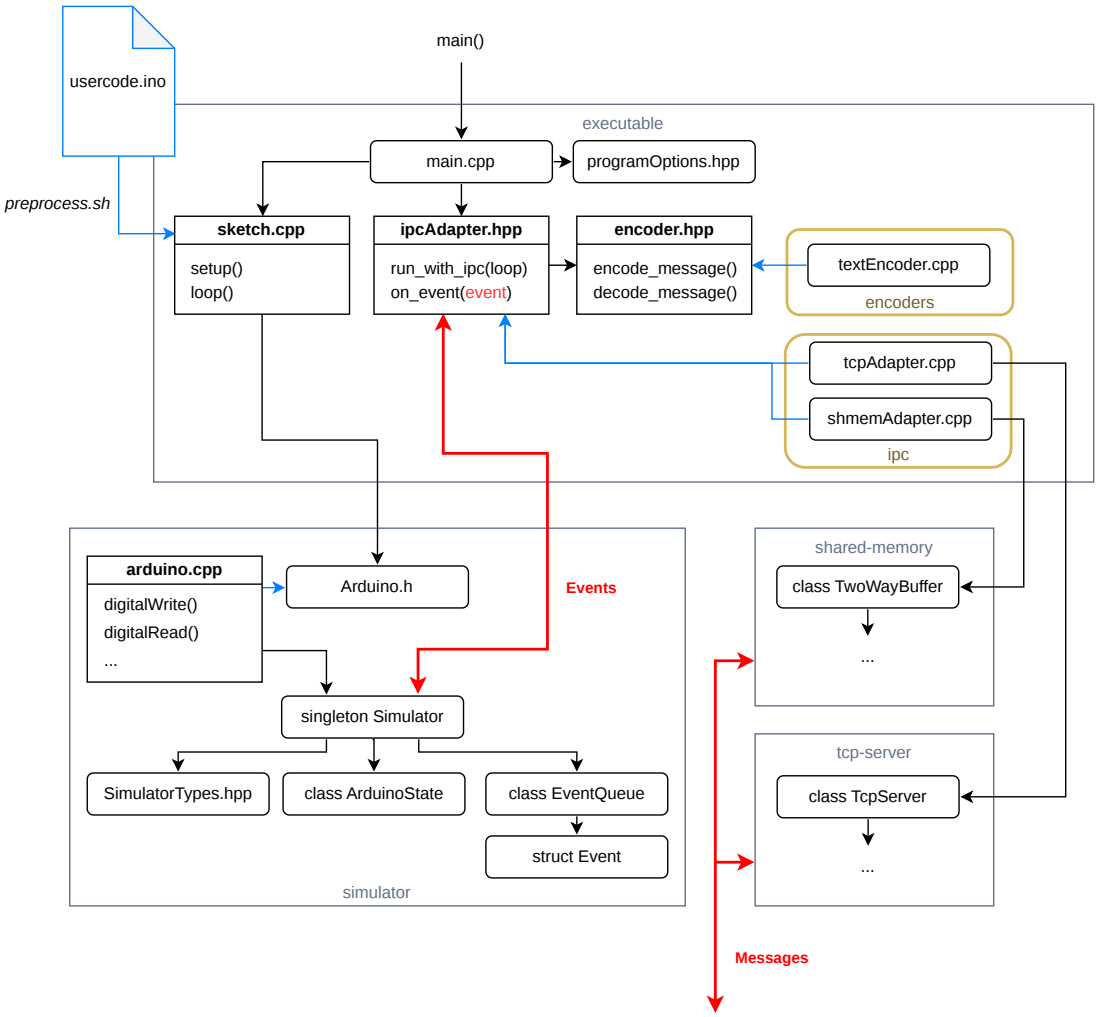

Figure 2.3 Backend architecture

2.3 Backend

2.3.1 The simulation

The language choice for backend was obvious, the `.ino` file is a syntactically valid C++ code. We include the code in the project by passing it as an argument to the `preprocess.sh` script (`preprocess.bat` on Windows). Inside we use the `arduino-cli` tool with the right arguments to apply very light prerpocessing to the file, like adding function declarations to the top or including the `Arduino.h` header. Important additions are the `#line` directives, which link to the definitions in the original `.ino` file. Thanks to these, breakpoints can be added to the `.ino` file and debugged as if the file itself was part of the project, completely with stepping through lines and inspection of variables. The file is then saved as `sketch.cpp` in the directory of the `executable` module where it is picked up by the CMake configuration and included in the `main.cpp`. The functions declared by the `Arduino.h` header file, like `digitalWrite` or `millis` are implemented in a custom mock library `Arduino.cpp` in the `simulator` module. Instead of instructing the AVR chip, as the real implementation would, the mock definitions create events and pass them to the `Simulator` for enquing and consumption.

The `Simulator` singleton class is the beating heart of the backend. It is responsible for handling the events and the event queue, managing the Arduino state, logging changes and keeping track of time. It is initialized in `main.cpp` and used frequently throughout the `executable` and `simulator` modules. The `Simulator` holds the instance of the `ArduinoState` class, representing the digital states on its pins, current pin modes and the inner timer of the Arduino. It also holds the instance of the `EventQueue`, which stores both local and remote events waiting for consumption. The `SimulatorTypes.hpp` is a header file with very few type defintions commonly used in the `simulator` module.

2.3.2 Events and IPC

The `Event` struct is just a "recipe" for what changes should the `Simulator` do to the `ArduinoState`. It holds a lambda function, corresponging to an event's type that is executed only after the event is put in the event queue and consumed in the order described in the section 2.2.3. The struct defines static functions that create new event instances with these predefined lambdas with the event type's parameters. When a local event is consumed, i.e. the lambda is executed, the event is signalled back to the `executable` module through a callback, that has been passed to the `Simulator` during initialization. This way the event can be sent to an IPC adapter, regardless of the used implementation, without a circular dependency on the `executable` module.

The IPC adapter itself is an interface of two functions declared in the `ipcAdapter.hpp`. `run_simulator_with_ipc` initializes the IPC mechanism. It takes the simulator loop function as an argument and executes it after IPC has been set up. That is because the IPC usually needs to create new threads that cannot be joined before the loop stops running. The loop function itself is defined in `main.cpp` and it contains both the `setup()` and `loop()` calls from the user's code.

The `on_event` function of the adapter is the callback passed to the `Simulator`. It encodes the event into a message and sends over IPC to the remote process. Listening to events of the remote process occurs on a different thread than the one with the simulator loop. Incoming messages are decoded into `Event` structs and sent directly to the event queue in the `Simulator`. `encode_message` and `decode_message` are functions declared in yet another interface in the `encoder.hpp` header. It's implementations are in the `encoders` directory, i.e. only `textEncoder.cpp` at the moment.

Implementations of the IPC adapter are located in the `ipc` directory. The `tcpAdapter.cpp` depends on the `tcp-server` module, while the `shmemAdatper.cpp` depends on the `shared-memory` module. Which one is used depends on the chosen compilation target. The `tcp-server` module contains definition of the TCP server and mechanisms for accepting new connections and handling the stream of data using the `boost::asio` library. The `shared-memory` module contains memory mapping mechanisms using the `boost::interprocess`, implementation of the circular buffer and the producer-consumer pattern using two buffers.

2.3.3 Utilities

Logger

There are additional utility modules not shown in the figure 2.3. One of them is the humble `logger` used extensively throughout the project. The `ILogger` abstract class exposes the `log` function accepting either a raw string or an `IMessage` abstract struct. Structs inheriting form `IMessage` represent unified formatting of a message with a timestamp and a level of verbosity. The verbosity levels from the highest to lowest are *Debug*, *Info*, *Warning* and *Error*. The messages of higher verbosity levels can be ommitted from the logs by setting the appropriate command line option. `ILogger` offers shorthand functions for logging these general messages, but more specific ones can be defined as well.

The module comes with `FileLogger` implementation of the `ILogger`. It creates or opens a file and dumps the logs inside. To avoid race conditions while writing to the files, two of these loggers are used in the project. One is owned and used by the `Simulator` singleton for logging inner workings of the Arduino. The other is owned by the IPC adapter and used for logging the exchanged messages or potential failures of the IPC mechanisms.

Timer

The other utility model is `Timer`. It is used to track time from the start of the simulation. The simulator holds two of these timers, one for *real time* and one for *simulation time*. Although there is no difference between them right now, the intention is to apply effects like stopping or slowing down to the simulation time timer from the frontend, so that functions like `millis` work consistently with disturbances caused by the frontend. Functions returning real time and simulation time are passed to the file logger as a source of its timestamps.

2.4 Frontend

2.4.1 The presentation

The frontend is written in C#. We consider the language to be more convenient for writing many of the essential systems than its C++ counterpart. That includes handling the graphics, the user interactions and parsing numerous configuration files needed for extensibility of the application. Libraries are also much easier to set up and compile thanks to NuGet.

For the GUI we decided to use Avalonia UI. Avalonia is a popular open-source cross-platform .NET framework, used by major companies like JetBrains or Unity. It is based on the Microsoft's WPF framework utilising the same XAML markup language and many of the same core principles. We chose it exactly because neither WPF or its cross-platform successor MAUI are available on Linux, one of our target platforms. We only include Avalonia in the Main project of the frontend, as this is the only project responsible for rendering.

Our humble GUI consists mostly of the circuit canvas. A zoomable and pannable space where the *components* of a circuit can be placed to specific coordinates, uncostrained by the usual layout structures of Avalonia. Arrangement

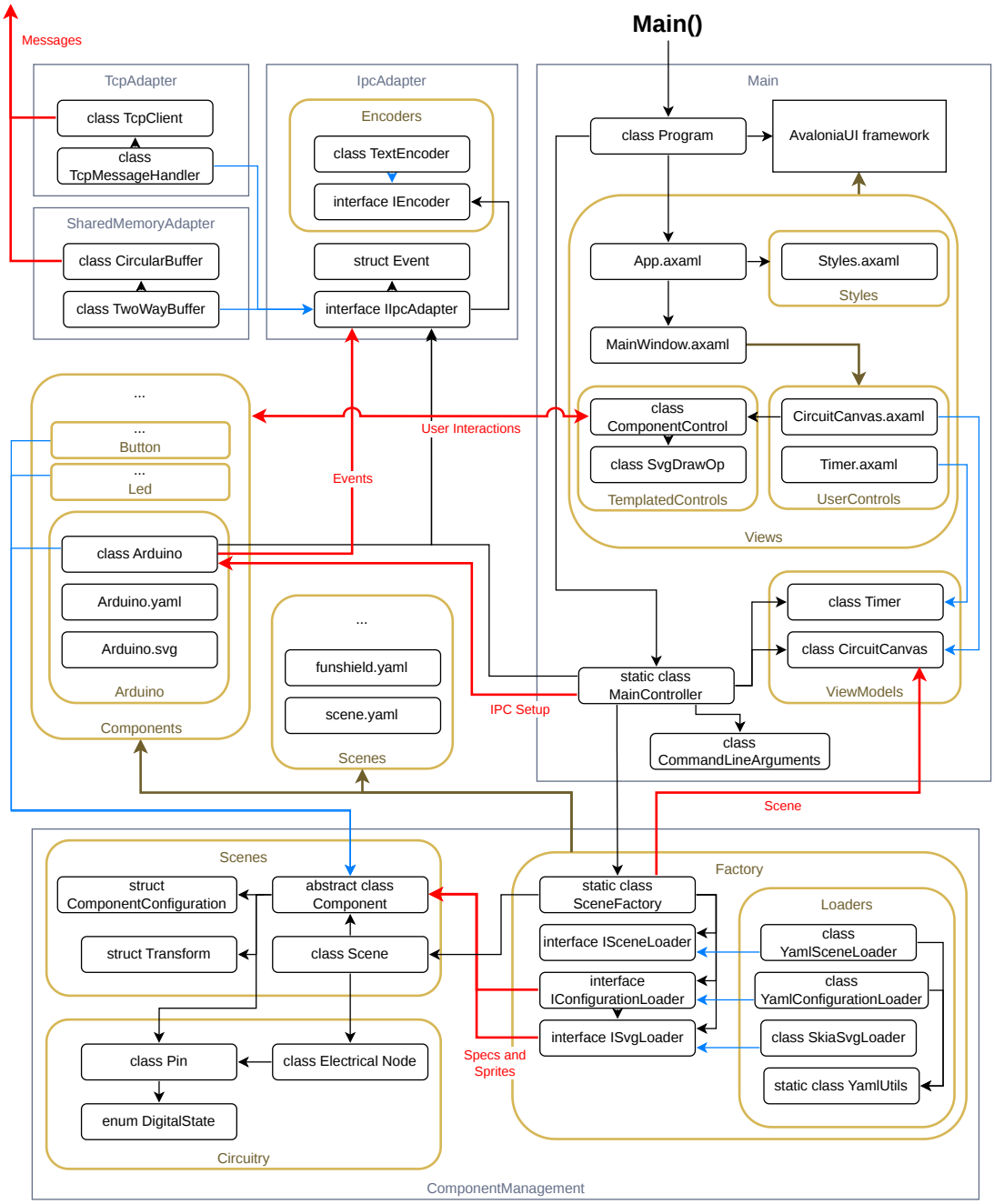

Figure 2.4 Frontend architecture

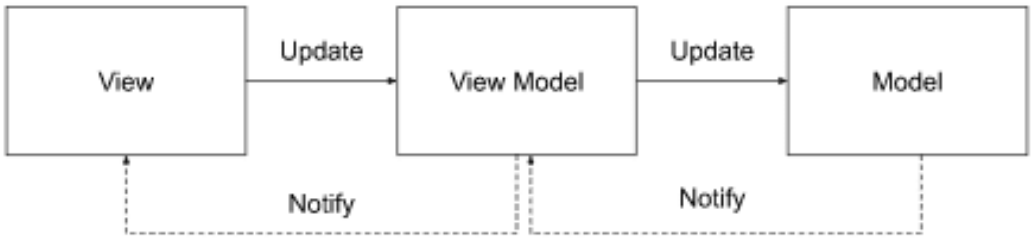

Figure 2.5 MVVM architecture [6]

of these components is called a *scene*. Each of the component types have its own set of SVG images that are drawn to the canvas. We refer to them as *sprites*. Each individual component always references one of its sprites, conveying the current state to the user. A sprite on the canvas can be clicked on (e.g. a button) to generate some kind of response. Of course, these features are not built into Avalonia, so we need to create custom Avalonia *controls* to achieve them.

Model-View-ViewModel

The preferred architecture for Avalonia applications is *MVVM*, or *Model-View-ViewModel*. *View* is the visual layout of the window and its parts. It is written in *XAML* (Extensible Application Markup Language) in files with the `.axaml` extension. In Avalonia's case, elements of a view are called *controls*. Those can be buttons, labels, containers, etc. 3 types of custom controls can be created in total, our frontend utilizes *user controls* and *custom-drawn controls*. User controls are themselves views built up in XAML from other controls. This is perfect for our `CircuitCanvas` view, as it is just a list of component controls enveloped in a built-in `Canvas` panel. The custom-drawn controls are defined as C# classes, inheriting from the `Control` class, and defining it's own rendering logic by overriding the `Render` method [5].

We need to implement the visuals of our components using custom-drawn controls, as there are no built-in or library controls that can draw a precompiled SVG image and swap it during runtime. Thus we define a `ComponentControl`, that calls a custom drawing operation in its `Render` method. That is defined by the `SvgDrawOp` class, which takes a sprite and a transformation matrix as arguments, applies visual transformations on the image and instructs the Avalonia's rendering pipeline to show it in the parent view.

XAML views bind to data and events in a *viewmodel*, which manipulates with data shown in the view. A model then is the raw data passed to the viewmodel, accessed by some database or a service, or produced by the business logic. View is aware of the viewmodel and viewmodel is aware of the model, but not vice versa. In ideal case, the model should be agnostic about the UI technology used. [6]

Our model is the abstract `Component` class implemented by various types of components, like LEDs, buttons or our Arduino. A list of the `Component` instances is stored in `CircuitCanvas` viewmodel, and each instance is passed to a `ComponentControl` in the `CircuitCanvas` view. This and all other initialization steps of the application are orchestrated by the static `MainController` class.

2.4.2 The logic

Only a minor part of the frontend is actually written with Avalonia. The business logic of the aforemention `Component` model is all encompassed in the `ComponentManagement` project. That means parsing of the components definition, creating instances of a scene, and the simulation of the circuit. This project is completely unaware, that it will be rendered with Avalonia specifically.

The circuit can be thought of as an unoriented multigraph, where the nodes of the graph are both *electrical nodes* and the *components*. All neighbors of an electrical node are components, and all neighbors of a component are electrical nodes. An edge of the graph represents a conductive connection between an electrical node and a *pin* of a component. Components usually have multiple pins.

In a digital circuit, an electrical node always obtains one of two digital states, *high* or *low*, which is given by states of pins it is connected to. When a high state is written into a pin, we say it is *driving*. When a low state is written into a pin, it becomes not driving again. The digital state of an electrical node is high precisely when at least one of the pins it is connected to is driving. A change in the state of an electrical node is propagated through all of the connected pins. Components can react to this change by writing new states to their other pins creating a chain reaction of state changes in the circuit.

Components

The frontend's user is encouraged to extend the simulation with their own implementations of the abstract `Component` class. That is done by creating a new *component directory* with the name of the component. The shared path to all component diretories is specified as one of the command line options of the frontend. The static information about the component type is stored as a record `ComponentConfiguration`, created from the files in the directory. The required files are a YAML file with metadata and at least one SVG file for the sprite. The YAML file contains in the very least a name to identify the component type, and a list of the component's pins. The pins can be defined either as an enumeration of their names, or as a single integer, that represents the total count of unnamed pins identified by their index.

The `Component` base class provides multiple virtual methods with the `On*` prefix. These are called upon changes in the circuit or as a result of a user interaction. The most important one is the `OnPinStateChanged` with the `Pin` object passed as an argument. This is the primary way of defining the component's logic. `OnControlPress` and `OnControlRelease` are other useful methods for interactible components, such as buttons. The base method `UpdateSprite` changes the rendered sprite to a different one, identified by the name of its SVG file. The base `GetPin` method returns a `Pin` object of the component's instance identified either by its name or index. Digital states are read and written through this object.

Usually, specific pins have inherent semantics of being used only for reading or only for writing. This property can be set explicitly on the `Pin` object with `MakeReadOnly` and `MakeWriteOnly` methods. Read only pins refuse write attempts from the component's logic and write only pins refuse notifying a component with a new state from node's propagation. General pins, defined by method `MakeGeneral`

don't have these constraints. Incorrect use of pins with these semantics is followed by a warning in the simulation logs.

Listing 2.1 Component configuration example: Led.yaml

```yaml
name: Led
description: Light emitting diode
imageSize: 50 50
pinNames:
  - anode
  - cathode
```

Scenes

The final form of the graph is achieved with creation of the scene. The YAML file of the scene consists of two sections. In the `Components` section, individual instances of `Component` subtypes are defined. They must have a unique name, and they should specify a graphic transformation, i.e. position, rotation and scale. This triplet of 2D vectors is composed into a `Transform` struct and passed into the `SvgDrawOp` as a transformation matrix. The `Nodes` section defines individual electrical nodes. A single node in this file is represented as a list of component-pin pairs, where the components are identified by their instance name in the `Components` section, and the pin is identified by its definition in the `ComponentConfiguration` of the given `Component` subtype.

The creation of the scene is comprised of a series of parsers in the `Factory` directory. There is an interface for each of the file types, i.e. the scene, component configurations and SVG files. Each has exactly one implementation in the `Loaders` directory, using `YamlDotNet` and `SvgSkia` libraries. They are put together in the static `SceneFactory` class, where component subtypes and instances are finalized. The results is a complete `Scene` object with initialized `Component` instances and `ElectricalNode` lists. The `SceneFactory` is called by the `MainController` in the `Main` project, which passes the scene to the `CircuitCanvas` viewmodel.

Listing 2.2 Scene example: minimal.yaml

```yaml
Components:
  Led:
    Led1:
      Position: 100 100
    Led2:
      Position: 200 100
  Button:
    Btn:
      Position: 100 200
  Vcc:
    Source:
      Position: 125 300
Nodes:
  - [Source.0, Btn.0]
  - [Led1.cathode, Btn.1]
  - [Led2.cathode, Btn.1]
```

The example scene in the listing 2.2 contains 2 LEDs, 1 button and 1 source of voltage. Source, which has one pin that is always driving, is connected to the 0th pin of the button, which reads a high value of the node. Both LEDs are connected

to the 1st pin of the same button. Pressing the button will copy the digital state from pin 0 to pin 1, and thus lights up the LED.

2.4.3 The Arduino component and IPC

Arduino is one of the component subtypes of the `ComponentManagement` project. It is a partial class divided into 2 files. In `Arduino.cs` it overrides virtual methods and deals with interaction with the circuit. In `Arduino.Events.cs` it creates events and sends to IPC. It also calls its own method from `Arduino.cs` when a new event arrives, based on its type and arguments. The adapter is now defined as an `IIpcAdapter` interface, which is owned by the Arduino component. Events are exchanged through this interface.

The interface is defined in the `IpcAdapter`, and is implemented by the `TcpAdapter` and `SharedMemoryAdapter` projects. It declares methods for connecting to IPC and sending messages. It also declares a C# event called `ReceivedEventEvent`, which is invoked after decoding a message incoming from the backend, with the decoded `Event` struct as an argument. The Arduino component is subscribed to `ReceivedEventEvent`.

The `IpcAdapter` project also contains message encoders and a definition of the `Event` struct. The C++ and the C# version of the `Event` struct have the same signature, with the exception of owning a lambda function (described in 2.3.2). The lambda is not necessary for the frontend, as there will be only one source of events executed in the frontend, the ones received by the backend. They don't have to be put in a queue, they can be consumed immediately as they come. Frontend also produces events, e.g. when value on an Arduino's input pin is changed from the circuit, but those are never consumed by the frontend. It is the reponsibility of the backend to handle them.

`IEncoder` and its `TextEncoder` implementation are analogous to `encoder.hpp` and `textEncoder.cpp` in the backend. The interface declares `EncodeEvent` and `TryDecodeEvent`, which transcribe events structs into human-readable strings and vice versa.

2.4.4 Utilities

Logger

The `Logger` project is analogous to its backend counterpart (2.3.3). It provides logging to a file with 4 levels of verbosity, that can be set up with a command line argument. Two loggers are used in the frontend as well, one for IPC and one for circuitry logs. There is one additional implemenation of the `ILogger` called `CompositeLogger`, which holds list of loggers, and logs into all of them simultaneously. This mechanism anticipates a need for another logger implementation, which will display the log messages in the GUI using Avalonia.

3 Backend implementation

3.1 Simulator module

3.2 Executable module

3.3 TCP Adapter module

3.4 Shared Memory Adapter module

3.5 Utilites

4 Frontend implementation

4.1 Component Management project

4.2 Main project with Avalonia UI

4.3 IPC Adapter project

4.4 TCP Adapter project

4.5 Shared Memory Adapter project

4.6 Logger project

5 User documentation

5.1 Compiling and running the simulator with Arduino code

5.1.1 Build process and usage examples

First make sure you comply with the software prerequisits from the section 1.4. Make sure your Boost libraries can be found by the CMake metabuild system from your development environment. For further information, please follow the Boost's getting started guide [7]. After that, we can preprocess the sketch, compile it with the simulator, and run the simulator together with the frontend.

Backend process

1. Run `preprocess.sh` with the sketch directory. On Windows, use the `preprocess.bat` script instead. For example:

```
./preprocess.sh tests/buttons.
```

2. Build `backend/CMakeLists.txt`. Available targets are:

   `arguino-shared-memory.exe` for IPC over shared memory

   `arguino-tcp.exe` for IPC over TCP connection

   `arguino` is an alias for the shared memory version (Default and preferred version).

3. Run

```
<build dir>/executable/<build target>
```

Frontend process

1. Build `frontend/gui.sln`.

2. Run the `Gui` executable with the desired scene. For example, in the `frontend` directory call:

```
<build dir>/Gui.exe -s ./Scenes/buttons.yaml.
```

   If you built the TCP version of the backend, be sure to add the flag `-i Tcp`.

For frontend and backend to communicate properly, it is necessary that they connect to the same IPC object. For TCP it is the port (`-p` option for both programs), and for shared memory it is the shared memory's object name (`--shmem-name` option for both programs). These default to the same value for both of the aforementioned options, port `8888` and memory named `Arguino-ipc`.

5.1.2 Command-Line Arguments

Listed below are all the available command line options for the backend and frontend executables. Default values, if defined, are specified in the square brackets. Expected arguments are in the angle brackets. For arguments accepting only specific enumeration of values, all available values are listed in the angle brackets seperated by a vertical bar (`|`). In square brackets is the default value used, when the option is omitted.

Backend

`--log-simulator <path>` Path to the log file for simulator events. [`./core.log`]

`--log-ipc <path>` Path to the log file for Inter-Process Communication (IPC). [`./core_ipc.log`]

`-v, --verbosity <Debug | Info | Warning | Error>` Verbosity level of the log files. [`Info`]

Frontend

`-s, --scene <path>` Path to the `.yaml` scene definition file. [`./scene.yaml`]

`-c, --components <path>` Path to the directory containing component definitions. [`./ComponentManager/Components`]

`-i, --ipc <None | Tcp | Shmem>` Inter-process communication technology utilized for connecting with the simulator. [`Shmem`]

`--log-circuit <path>` Path to the log file for general circuitry events. [`./frontend.log`]

`--log-ipc <path>` Path to the log file for IPC events. [`./frontend_ipc.log`]

`-v, --verbosity <Debug | Info | Warning | Error>` Verbosity level of the log files. [`Info`]

Common for TCP usage

`-p, --port <int>` TCP port to listen to. [`8888`]

Common for shared memory usage

`--shmem-name <string>` Name of the memory-mapped file. [`Arguino-ipc`]

`--shmem-size <int>` Size of the memory-mapped region per buffer, specified in pages. [`1`]

5.2 Defining components and scenes

Extensibility of the frontend through custom components and circuits is an important part the project. Here are necessary steps and requirements to create and use new components.

5.2.1 Required file structure

All of the components must reside in one directory, referred to as *definitions directory*. Each component is represented by exactly one directory, referred to as *component directory*, placed in the definitions directory. The definitions directory must be supplied through the `-c` command line argument. These files must be present in the component directory:

- C# behavior script
- YAML component configuration
- One or more .svg images

The yaml file must have the same name as the component directory, i.e. `MyComponent/MyComponent.yaml`. The SVGs will be referenced in behavior script by their file name, excluding the .svg extension. For example `image.svg` will be referred to as `image`.

Notes on used YAML notation

In the next subsections, we will describe the expected structures of the YAML files using this notation:

- angle brackets must be replaced by proper parameters, i.e. `name: <string>`
- properties with question mark are optional, i.e. `optional?: ...`
- Vector2 is represented as a string of 2 float values separated by a space, i.e. `vector: 7.27 6.9`

5.2.2 Component configuration file specification

This file defines common properties of the custom component type.

Listing 5.1 YAML configuration file structure

```yaml
name: <string>
description?: <string>
imageSize?: <Vector2>
pins?: <uint>
pinNames?:
    - <string>
    - ...
    - ...
```

`name` Name of the component type as displayed in the UI. (not used yet)

`description` Brief description of the component as displayed in the UI. (not used yet)

`imageSize` Desired size of the component's sprites in pixels for default zoom level of the canvas. Defaults to <50, 50>.

pins Number of unnamed pins. These pins are identified by their 0-based index in both the scene definition and in the component logic.

pinNames List of unique names for each pin. These pins are identified by their given name in both the scene defintion and in the component logic.

5.2.3 Component behavior script specification

How does the component interact with the rest of the circuit is described by the C# class with the named the same as the component type. This class must inherit from the abstract `Component` class. The file containg the class must be added to the `ComponentManagement.csproj` project. In order to compile, the user's class must also define a constructor like this:

```
1    public MyComponent(string definitionPath)
2    : base(definitionPath) { ... }
```

However, no code is required in the body of the constructor. The constructor can be used for initialization of properties known before the scene is created. The `Component` class provides most of the necessary API.

Component functions and properties

virtual void OnInitialized() Gets called when all of the component instances have been initialized and scene created.

virtual void OnPinConnected(Pin) Gets called when a component's pin connects to an electric node. The connected pin is passed as argument.

virtual void OnPinDisconnected(Pin) Gets called when a component's pin disonnects from an electric node. The disconnected pin is passed as argument.

virtual void OnPinStateChanged(Pin) Gets called when a component's pin detected a change in digital state of a node it has been connected to. It can be thought of as changing input value on the pin. Pins set to WriteOnly mode by design don't detect this change. The detected pin is passed as argument.

virtual void OnControlPress(Vector2) Gets called when the component has been pressed on the circuit canvas. The cursor position at the time of the press is passed as argument.

virtual void OnControlRelease Gets called when the component has been released on the circuit canvas.

void UpdateSprite(string) Changes the .svg image drawn to the canvas. The new image is given by the svg name specified in the argument.

Pin? GetPin(string) Returns a Pin object with the name given in the argument. The name must be defined in the component's `namedPins` list. Returns null if no such pin exists.

`Pin? GetPin(uint)` Returns a Pin object with the id given in the argument. The allowed range of Ids is from `0` to $pins + namedPins.Length$. With unnamed pins starting from the id `0` and named pins starting from the id `pins`. Returns null if the pin id is out of range.

Pin functions and properties

`uint Pin.Id` Id of the pin, as described in the `GetPin(uint)` section.

`string Pin.Name` Name of the pin as described in the component configuration.

`bool Pin.IsDriving` True if the pin is driving. That is, if it actively forces the digital state on the connected node to be high. If the node has at least 1 driving pin, it's digital state will be high. It will be low otherwise.

`bool Pin.IsHigh` True if the digital state of the connected node is high.

`bool Pin.IsLow` True if the digital state of the connected node is low.

`bool Pin.IsWriteOnly` Write only pins don't react to changes in the connected node's digital state.

`bool Pin.IsReadOnly` Read only pins cannot be set to driving.

`bool Pin.IsConnected` Whether is the pin connected to any electrical node.

`void Pin.MakeReadOnly()` Sets the pin to read only mode. Read only pins cannot be set to driving.

`void Pin.MakeWriteOnly()` Sets the pin to write only mode. Write only pins don't react to changes in the connected node's digital state.

`void Pin.MakeGeneral()` Sets the pin to general mode. General pins don't pose any restrictions to reading or writing.

`void Pin.SetValue(bool)` If the argument is true, it sets the pin to be driving. Otherwise, the pin set to not drive.

`void Pin.SetHigh()` Sets the pin to be driving. Same as `SetValue(true)`.

`void Pin.SetLow()` Sets the pin to be not driving. Same as `SetValue(false)`.

5.2.4 Defining scenes

The scene YAML file describes component instances present in the scene, and how are their pins connected together in nodes. The path of this file is supplied through the `-s` command line argument. There are no restriction to where the scene file has to be placed.

Listing 5.2 YAML scene file structure

```yaml
components:
  <Type name>:
    <Instance name>:
      position: <Vector2>
      rotation?: <float>
      scale?: <Vector2>
    <Instance name>:
      ...
nodes:
  - [ <Instance name>.<Pin name>, ...]
  - ...
  - ...
```

components Dictionary of component type objects. These objects are dictionaries of component instances of that type. These instances define properties of the individual components.

Type name String. Name of the component type. This must match the component directory name.

Instance name String. Name of the component instance. It identifies the instance within the node definition and in the UI. It must be unique across all instances within the components dictionary.

position Coordinates on the circuit canvas, with origin in the upper left corner.

rotation Clockwise rotation angle in degrees on the circuit canvas.

scale Relative scaling along x an y axis on the circuit canvas.

nodes List of lists. The inner list represents all pins connected to the same electrical node. Its element is an instance-pin pair separated by a dot. Instance name must be present in the components dictionary. Pin name must be present in the component's configuration. In the case of unnamed pins, pin name is simply its index.

5.3 IPC interface

This section describes how to set up IPC communication of a custom frontend application with the backend simulator. The two processes can communicate together either via TCP connection or through shared memory. The only requirement for a TCP connection, is that the client on the frontend listens to the same port as the server on the backend. IPC over shared memory requires a few more considerations, so that the memory accesses match the memory layout of shared internal structure created by the backend. This structure is described in the subsection 5.3.3;

The processes communicate using concrete events with up to 3 integer arguments. These events are encoded into a human readable text messages, seperated by a common delimeter. The messages are then sent over IPC.

5.3.1 Text message format

Individual messages are delimited by the semicolon character (`;`). The individual values within one message are delimited by the colon character (`:`). The integer values are written in decimal system, using the ASCII characters of the actual digits. The semantics of these values are better described in the chapter about architecture, subsection 2.2.3. The format of the message then looks as the following:

Listing 5.3 Format of an IPC text message

```
<sim_time>:<lvt>:<event_type>:<arg1>:<arg2>:<arg3>;
```

`sim_time` Simulation time, 64-bit signed integer. Real time elapsed from the start of the backend's simulation represented in microseconds. The backend has no use for this value.

`lvt` Local virtual time, 64-bit signed integer. It describes an order in which the event occured, relative to other events.

`event_type` A single character representing the type of the event. This type decides the action taken upon consuming the event. Recognized event types and their corresponding characters are described in the next subsection.

`arg1, arg2, arg3` 32-bit signed integers. Values have meaning depending on the event. Arguments not defined by the event type can be omitted. Make sure that the number of colons match the number of passed arguments.

For example, this next message encodes the `Write` event passing 2 arguments at the simulation time of 264 seconds:

Listing 5.4 Example of an IPC text message

```
000264997488:0000016:W:08:1;
```

Decide on whether to describe or remove the zero paddings.

5.3.2 Recognized event types

Write event

Signalizes digital value being written into a pin. If consumed by the backend, the value gets written to the internal `ArduinoState`. If consumed by the frontend, it should reflect the changes caused by the change of the pin's state. This event is produced by the backend on `digitalWrite` function call within the sketch.

Encoding character: `W`

Arguments:

1. **Pin index** of the digital pin from the range 0–13.
2. **Value** 0 for the low value, 1 for the high value.

Pinmode event

Signalizes change of a pin mode of a pin. If consumed by the backend the value gets written to the internal `Arduinostate`. If consumed by the frontend, it should change the behavior of the pin, such that only input pins are able to send the `Write` event. This event is produced by the backend on `pinMode` function call within the sketch.

Encoding character: `P`

Arguments:

1. **Pin index** of the digital pin from the range 0–13.
2. **Mode** 0 for the output mode, 1 for the input mode.

Reboot event

Produced by the backend, when the simulation has started over. Useful, when the backend process has be terminated and then executed again. At this point the frontend has to invalidate its current state and start again. Backend itself does no action upon consuming this event (it may force the reboot of the simulation in the future).

Encoding character: `R`

Arguments: None

5.3.3 Shared memory specification

Locating memory mapped file

On Linux, the mapped memory region is orchestrated by a proper shared memory object. It can be accessed either by a syscall, or by reading the `\dev\shm\<shmem_name>` virtual file.

On Windows, the shared memory is emulated by the `Boost.interprocess` library via a memory mapped file. It can be located at the path specified by the `Common AppData` registry. For Windows Vista and newer, this path defaults to `C:\ProgramData`. On this path, a `boost_interprocess` directory is created. The memory mapped file, named the same as the shared memory object, can be found in one if its subdirectories. [8]

In either case, the name of the read memory mapped file or shared memory object has to match the name given to the mapped region via the `--shmem-name` flag of the backend command.

Memory layout

The mapped memory region is split into 2 equal sections, both containg the same circular buffer data structure. So for a mapped region of size n bytes, and for the start address of the region s_a, the second buffer is located on the memory address $s_a + n/2$ (n is always even, as it is a multiple of a page size). Let's denote the start address of a buffer s_b

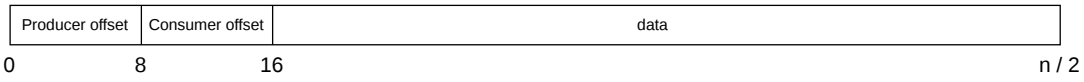

Figure 5.1 Memory layout of the circular buffer

The buffer itself has 2 control variables stored within it. Both are 64-bit unisgned integers representing an offset within the buffers data. The first, located at s_b, is the offset at which the producer will write the next chunk of data into the buffer. The second, located at $s_b + 8$, is the offset at which the next chunk of data will be read from the buffer, if present. From the address $s_b + 16$ until the next buffer, or the end of the mapped region, the actual data are stored. That means the actual capacity of data stored within one buffer is $n/2 - 16$.

The data have no uniform size, structure or address alignment. The length of a message in the buffer must be deduced from the data within the message itself. In case of the text encoding, it is the message delimeter. A message can be located at the boundary of the data region. In that case the message is split so that very byte, until the very end of the region is filled, and the rest is written at $s_b + 16$ again. There are no gaps between the messages.

The frontend process is responsible for consuming messages from the first buffer and producing them to the second one. As such, the consumer offset of the first buffer and the producer offset of the second buffer must be updated accordingly. The other two offsets must remain read only. The backend process has these roles reversed.

Conclusion

In the end we managed to develop two nontrivial applications. One for running and simulating the user's Arduino code. The other one for creating a interactive virtual environment, where the Arduino can be connected into a circuit with visual feedback. In chapter 2 of this thesis we described the thought process, project structure, and technologies used in order to achieve the goals we set for ourselves in section 1.4. Then we looked into more detail on how we implemented these technologies while avoiding many pitfalls in chapters 3 and 4.

While we implemented only a couple of most used functions from the Arduino library, and the user interface leaves a lot to be desired, we believe the current version of the product represents a solid foundation on which new functions, events, components and quality of life features can be laid down. One of the biggest hurdles was making sure that the two applications stay synchronized. We succeeded in that regard, by learning more of useful information about interprocess communication and approaches commonly used in distributed computing.

A big success also lies in the extensive component system developed in the frontend. The user can extend their component library just by following the specification in section 5.2. They can use custom vector graphics, and give life to their components thanks to the abstract `Component` class. Then they can create it into a scene together with Arduino running their code. All this while all the heavy work is done by the reflection and parsing machinery on the background. We would be excited to work on this component system, so that it brings even more possiblities to the user.

Bibliography

1. *Sketch specification | Arduino Documentation* [online]. [visited on 2026-06-25]. Available from: https://docs.arduino.cc/arduino-cli/sketch-specification/.
2. *UNO R3 SMD | Arduino Documentation* [online]. [visited on 2026-06-25]. Available from: https://docs.arduino.cc/hardware/uno-rev3-smd/.
3. *digitalRead() | Arduino Documentation* [online]. [visited on 2026-06-25]. Available from: https://docs.arduino.cc/language-reference/en/functions/digital-io/digitalread/.
4. FUJIMOTO, R.M. Parallel and distributed simulation systems. In: *Proceeding of the 2001 Winter Simulation Conference (Cat. No.01CH37304)* [online]. Arlington, VA, USA: IEEE, 2001, pp. 147–157 [visited on 2026-03-05]. ISBN 978-0-7803-7307-5. Available from DOI: 10.1109/WSC.2001.977259.
5. *Choosing a custom control type | Avalonia Docs* [online]. 2026-04-20. [visited on 2026-05-19]. Available from: https://docs.avaloniaui.net/docs/custom-controls/choosing-a-custom-control-type.
6. *The MVVM pattern | Avalonia Docs* [online]. 2026-04-20. [visited on 2026-05-19]. Available from: https://docs.avaloniaui.net/docs/fundamentals/the-mvvm-pattern.
7. *Boost User Guide - Getting started* [online]. [visited on 2026-06-27]. Available from: https://www.boost.org/doc/user-guide/getting-started.html.
8. *Sharing memory between processes* [online]. [visited on 2026-06-29]. Available from: https://www.boost.org/doc/libs/latest/doc/html/interprocess/sharedmemorybetweenprocesses.html.

List of Figures

List of Tables

List of Abbreviations

A Attachments

A.1 First Attachment